JavaScript is disabled on your browser.

compact1, compact2, compact3

java.text

Class NumberFormat

java.lang.Object
    java.text.Format
        java.text.NumberFormat

All Implemented Interfaces:

`Serializable`, `Cloneable`

Direct Known Subclasses:

`ChoiceFormat`, `DecimalFormat`

```
public abstract class NumberFormat
extends Format
```

`NumberFormat` is the abstract base class for all number formats. This class provides the interface for formatting and parsing numbers. `NumberFormat` also provides methods for determining which locales have number formats, and what their names are.

`NumberFormat` helps you to format and parse numbers for any locale. Your code can be completely independent of the locale conventions for decimal points, thousands-separators, or even the particular decimal digits used, or whether the number format is even decimal.

To format a number for the current Locale, use one of the factory class methods:

```
myString = NumberFormat.getInstance().format(myNumber);
```

If you are formatting multiple numbers, it is more efficient to get the format and use it multiple times so that the system doesn't have to fetch the information about the local language and country conventions multiple times.

```
NumberFormat nf = NumberFormat.getInstance();
for (int i = 0; i < myNumber.length; ++i) {
    output.println(nf.format(myNumber[i]) + "; ");
}
```

To format a number for a different Locale, specify it in the call to `getInstance`.

```
NumberFormat nf = NumberFormat.getInstance(Locale.FRENCH);
```

You can also use a `NumberFormat` to parse numbers:

```
myNumber = nf.parse(myString);
```

Use `getInstance` or `getNumberInstance` to get the normal number format. Use `getIntegerInstance` to get an integer number format. Use `getCurrencyInstance` to get the currency number format. And use `getPercentInstance` to get a format for displaying percentages. With this format, a fraction like 0.53 is displayed as 53%.

You can also control the display of numbers with such methods as `setMinimumFractionDigits`. If you want even more control over the format or parsing, or want to give your users more control, you can try casting the `NumberFormat` you get from the factory methods to a `DecimalFormat`. This will work for the vast majority of locales; just remember to put it in a `try` block in case you encounter an unusual one.

NumberFormat and DecimalFormat are designed such that some controls work for formatting and others work for parsing. The following is the detailed description for each these control methods,

setParseIntegerOnly : only affects parsing, e.g. if true, "3456.78" → 3456 (and leaves the parse position just after index 6) if false, "3456.78" → 3456.78 (and leaves the parse position just after index 8) This is independent of formatting. If you want to not show a decimal point where there might be no digits after the decimal point, use setDecimalSeparatorAlwaysShown.

setDecimalSeparatorAlwaysShown : only affects formatting, and only where there might be no digits after the decimal point, such as with a pattern like "#,##0.##", e.g., if true, 3456.00 → "3,456." if false, 3456.00 → "3456" This is independent of parsing. If you want parsing to stop at the decimal point, use setParseIntegerOnly.

You can also use forms of the `parse` and `format` methods with `ParsePosition` and `FieldPosition` to allow you to:

- progressively parse through pieces of a string
- align the decimal point and other areas

For example, you can align numbers in two ways:

1. If you are using a monospaced font with spacing for alignment, you can pass the `FieldPosition` in your format call, with `field` = `INTEGER_FIELD`. On output, `getEndIndex` will be set to the offset between the last character of the integer and the decimal. Add (desiredSpaceCount - getEndIndex) spaces at the front of the string.
2. If you are using proportional fonts, instead of padding with spaces, measure the width of the string in pixels from the start to `getEndIndex`. Then move the pen by (desiredPixelWidth - widthToAlignmentPoint) before drawing the text. It also works where there is no decimal, but possibly additional characters at the end, e.g., with parentheses in negative numbers: "(12)" for -12.

Synchronization

Number formats are generally not synchronized. It is recommended to create separate format instances for each thread. If multiple threads access a format concurrently, it must be synchronized externally.

See Also:

`DecimalFormat`, `ChoiceFormat`, `Serialized Form`

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
`static class`	**`NumberFormat.Field`**	Defines constants that are used as attribute keys in the `AttributedCharacterIterator` returned from `NumberFormat.formatToCharacterIterator` and as field identifiers in `FieldPosition`.

Field Summary

Fields

Modifier and Type	Field	Description
`static int`	**`FRACTION_FIELD`**	Field constant used to construct a FieldPosition object.
`static int`	**`INTEGER_FIELD`**	Field constant used to construct a FieldPosition object.

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	NumberFormat()	Sole constructor.

Method Summary

Abstract Methods

Concrete Methods

Modifier and Type	Method	Description
Object	clone()	Overrides Cloneable.
boolean	equals(Object obj)	Overrides equals
String	format(double number)	Specialization of format.
abstract StringBuffer	format(double number, StringBuffer toAppendTo, FieldPosition pos)	Specialization of format.
String	format(long number)	Specialization of format.
abstract StringBuffer	format(long number, StringBuffer toAppendTo, FieldPosition pos)	Specialization of format.
StringBuffer	format(Object number, StringBuffer toAppendTo, FieldPosition pos)	Formats a numbe and appends the resulting text to the given string buffer.
static Locale[]	getAvailableLocales()	Returns an array all locales for which the get*Instance methods of this class can return localized instanc

Currency	getCurrency()	Gets the currenc used by this number format when formatting currency values.
static NumberFormat	getCurrencyInstance()	Returns a curren format for the current default FORMAT locale.
static NumberFormat	getCurrencyInstance(Locale inLocale)	Returns a curren format for the specified locale.
static NumberFormat	getInstance()	Returns a genera purpose number format for the current default FORMAT locale.
static NumberFormat	getInstance(Locale inLocale)	Returns a genera purpose number format for the specified locale.
static NumberFormat	getIntegerInstance()	Returns an integ number format f the current default FORMAT locale.
static NumberFormat	getIntegerInstance(Locale inLocale)	Returns an integ number format f the specified locale.
int	getMaximumFractionDigits()	Returns the maximum numbe of digits allowed the fraction portion of a number.
int	getMaximumIntegerDigits()	Returns the maximum numbe of digits allowed the integer porti of a number.

int	**getMinimumFractionDigits**()	Returns the minimum numbe of digits allowed the fraction portion of a number.
int	**getMinimumIntegerDigits**()	Returns the minimum numbe of digits allowed the integer porti of a number.
static **NumberFormat**	**getNumberInstance**()	Returns a genera purpose number format for the current default **FORMAT** locale.
static **NumberFormat**	**getNumberInstance**(**Locale** inLocale)	Returns a genera purpose number format for the specified locale.
static **NumberFormat**	**getPercentInstance**()	Returns a percentage form for the current default **FORMAT** locale.
static **NumberFormat**	**getPercentInstance**(**Locale** inLocale)	Returns a percentage form for the specified locale.
RoundingMode	**getRoundingMode**()	Gets the **RoundingMode** used in this NumberFormat.
int	**hashCode**()	Overrides hashCode.
boolean	**isGroupingUsed**()	Returns true if grouping is used this format.
boolean	**isParseIntegerOnly**()	Returns true if th format will parse

		numbers as integers only.
Number	**parse**(**String** source)	Parses text from the beginning of the given string t produce a numbe
abstract **Number**	**parse**(**String** source, **ParsePosition** parsePosition)	Returns a Long i possible (e.g., within the range [Long.MIN_VALU Long.MAX_VALU and with no decimals), otherwise a Double.
Object	**parseObject**(**String** source, **ParsePosition** pos)	Parses text from string to produce a Number.
void	**setCurrency**(**Currency** currency)	Sets the currency used by this number format when formatting currency values.
void	**setGroupingUsed**(boolean newValue)	Set whether or n grouping will be used in this form
void	**setMaximumFractionDigits**(int newValue)	Sets the maximu number of digits allowed in the fraction portion o a number.
void	**setMaximumIntegerDigits**(int newValue)	Sets the maximu number of digits allowed in the integer portion o number.

void	**setMinimumFractionDigits**(int newValue)	Sets the minimum number of digits allowed in the fraction portion of a number.
void	**setMinimumIntegerDigits**(int newValue)	Sets the minimum number of digits allowed in the integer portion of a number.
void	**setParseIntegerOnly**(boolean value)	Sets whether or not numbers should be parsed as integers only.
void	**setRoundingMode**(**RoundingMode** roundingMode)	Sets the **RoundingMode** used in this NumberFormat.

Methods inherited from class java.text.Format

format, formatToCharacterIterator, parseObject

Methods inherited from class java.lang.Object

finalize, getClass, notify, notifyAll, toString, wait, wait, wait

Field Detail

INTEGER_FIELD

```
public static final int INTEGER_FIELD
```

Field constant used to construct a FieldPosition object. Signifies that the position of the integer part of a formatted number should be returned.

See Also:

FieldPosition, Constant Field Values

FRACTION_FIELD

```
public static final int FRACTION_FIELD
```

Field constant used to construct a FieldPosition object. Signifies that the position of the fraction part of a formatted number should be returned.

See Also:

FieldPosition, Constant Field Values

Constructor Detail

NumberFormat

```
protected NumberFormat()
```

Sole constructor. (For invocation by subclass constructors, typically implicit.)

Method Detail

format

```
public StringBuffer format(Object number,
                           StringBuffer toAppendTo,
                           FieldPosition pos)
```

Formats a number and appends the resulting text to the given string buffer. The number can be of any subclass of Number.

This implementation extracts the number's value using Number.longValue() for all integral type values that can be converted to long without loss of information, including BigInteger values with a bit length of less than 64, and Number.doubleValue() for all other types. It then calls format(long,java.lang.StringBuffer,java.text.FieldPosition) or format(double,java.lang.StringBuffer,java.text.FieldPosition). This may result in loss of magnitude information and precision for BigInteger and BigDecimal values.

Specified by:

format in class Format

Parameters:

number - the number to format

toAppendTo - the StringBuffer to which the formatted text is to be appended

pos - On input: an alignment field, if desired. On output: the offsets of the alignment field.

Returns:

the value passed in as toAppendTo

Throws:

IllegalArgumentException - if number is null or not an instance of Number.

NullPointerException - if toAppendTo or pos is null

ArithmeticException - if rounding is needed with rounding mode being set to RoundingMode.UNNECESSARY

See Also:

FieldPosition

parseObject

```
public final Object parseObject(String source,
                                ParsePosition pos)
```

Parses text from a string to produce a `Number`.

The method attempts to parse text starting at the index given by `pos`. If parsing succeeds, then the index of `pos` is updated to the index after the last character used (parsing does not necessarily use all characters up to the end of the string), and the parsed number is returned. The updated `pos` can be used to indicate the starting point for the next call to this method. If an error occurs, then the index of `pos` is not changed, the error index of `pos` is set to the index of the character where the error occurred, and null is returned.

See the `parse(String, ParsePosition)` method for more information on number parsing.

Specified by:

`parseObject` in class `Format`

Parameters:

`source` - A String, part of which should be parsed.

`pos` - A ParsePosition object with index and error index information as described above.

Returns:

A Number parsed from the string. In case of error, returns null.

Throws:

`NullPointerException` - if pos is null.

format

```
public final String format(double number)
```

Specialization of format.

Parameters:

number - the double number to format

Returns:

the formatted String

Throws:

ArithmeticException - if rounding is needed with rounding mode being set to RoundingMode.UNNECESSARY

See Also:

Format.format(java.lang.Object)

format

```
public final String format(long number)
```

Specialization of format.

Parameters:

number - the long number to format

Returns:

the formatted String

Throws:

ArithmeticException - if rounding is needed with rounding mode being set to RoundingMode.UNNECESSARY

See Also:

Format.format(java.lang.Object)

format

```
public abstract StringBuffer format(double number,
                                    StringBuffer toAppendTo,
                                    FieldPosition pos)
```

Specialization of format.

Parameters:

number - the double number to format

toAppendTo - the StringBuffer to which the formatted text is to be appended

pos - the field position

Returns:

the formatted StringBuffer

Throws:

ArithmeticException - if rounding is needed with rounding mode being set to RoundingMode.UNNECESSARY

See Also:

Format.format(java.lang.Object)

format

```
public abstract StringBuffer format(long number,
                                    StringBuffer toAppendTo,
                                    FieldPosition pos)
```

Specialization of format.

Parameters:

number - the long number to format

toAppendTo - the StringBuffer to which the formatted text is to be appended

pos - the field position

Returns:

the formatted StringBuffer

Throws:

ArithmeticException - if rounding is needed with rounding mode being set to RoundingMode.UNNECESSARY

See Also:

Format.format(java.lang.Object)

parse

```
public abstract Number parse(String source,
                             ParsePosition parsePosition)
```

Returns a Long if possible (e.g., within the range [Long.MIN_VALUE, Long.MAX_VALUE] and with no decimals), otherwise a Double. If IntegerOnly is set, will stop at a decimal point (or equivalent; e.g., for rational numbers "1 2/3", will stop after the 1). Does not throw an exception; if no object can be parsed, index is unchanged!

Parameters:

`source - the String to parse`

`parsePosition - the parse position`

Returns:

`the parsed value`

See Also:

`isParseIntegerOnly()`, `Format.parseObject(java.lang.String, java.text.ParsePosition)`

parse

```
public Number parse(String source)
             throws ParseException
```

Parses text from the beginning of the given string to produce a number. The method may not use the entire text of the given string.

See the `parse(String, ParsePosition)` method for more information on number parsing.

Parameters:

`source - A String whose beginning should be parsed.`

Returns:

`A Number parsed from the string.`

Throws:

`ParseException - if the beginning of the specified string cannot be parsed.`

isParseIntegerOnly

```
public boolean isParseIntegerOnly()
```

Returns true if this format will parse numbers as integers only. For example in the English locale, with ParseIntegerOnly true, the string "1234." would be parsed as the integer value 1234 and parsing would stop at the "." character. Of course, the exact format accepted by the parse operation is locale dependant and determined by sub-classes of NumberFormat.

Returns:

`true if numbers should be parsed as integers only; false otherwise`

setParseIntegerOnly

```
public void setParseIntegerOnly(boolean value)
```

Sets whether or not numbers should be parsed as integers only.

Parameters:

`value - true if numbers should be parsed as integers only; false otherwise`

See Also:

`isParseIntegerOnly()`

getInstance

```
public static final NumberFormat getInstance()
```

Returns a general-purpose number format for the current default `FORMAT` locale. This is the same as calling `getNumberInstance()`.

Returns:

`the NumberFormat instance for general-purpose number formatting`

getInstance

```
public static NumberFormat getInstance(Locale inLocale)
```

Returns a general-purpose number format for the specified locale. This is the same as calling `getNumberInstance(inLocale)`.

Parameters:

`inLocale - the desired locale`

Returns:

`the NumberFormat instance for general-purpose number formatting`

getNumberInstance

```
public static final NumberFormat getNumberInstance()
```

Returns a general-purpose number format for the current default `FORMAT` locale.

This is equivalent to calling `getNumberInstance(Locale.getDefault(Locale.Category.FORMAT))`.

Returns:

the NumberFormat instance for general-purpose number formatting

See Also:

`Locale.getDefault(java.util.Locale.Category)`, `Locale.Category.FORMAT`

getNumberInstance

`public static NumberFormat getNumberInstance(Locale inLocale)`

Returns a general-purpose number format for the specified locale.

Parameters:

`inLocale - the desired locale`

Returns:

the NumberFormat instance for general-purpose number formatting

getIntegerInstance

`public static final NumberFormat getIntegerInstance()`

Returns an integer number format for the current default `FORMAT` locale. The returned number format is configured to round floating point numbers to the nearest integer using half-even rounding (see `RoundingMode.HALF_EVEN`) for formatting, and to parse only the integer part of an input string (see `isParseIntegerOnly`).

This is equivalent to calling `getIntegerInstance(Locale.getDefault(Locale.Category.FORMAT))`.

Returns:

a number format for integer values

Since:

1.4

See Also:

`getRoundingMode()`, `Locale.getDefault(java.util.Locale.Category)`, `Locale.Category.FORMAT`

getIntegerInstance

`public static NumberFormat getIntegerInstance(Locale inLocale)`

Returns an integer number format for the specified locale. The returned number format is configured to round floating point numbers to the nearest integer using half-even rounding (see `RoundingMode.HALF_EVEN`) for

formatting, and to parse only the integer part of an input string (see isParseIntegerOnly).

Parameters:

inLocale - the desired locale

Returns:

a number format for integer values

Since:

1.4

See Also:

getRoundingMode()

getCurrencyInstance

```
public static final NumberFormat getCurrencyInstance()
```

Returns a currency format for the current default FORMAT locale.

This is equivalent to calling getCurrencyInstance(Locale.getDefault(Locale.Category.FORMAT)).

Returns:

the NumberFormat instance for currency formatting

See Also:

Locale.getDefault(java.util.Locale.Category), Locale.Category.FORMAT

getCurrencyInstance

```
public static NumberFormat getCurrencyInstance(Locale inLocale)
```

Returns a currency format for the specified locale.

Parameters:

inLocale - the desired locale

Returns:

the NumberFormat instance for currency formatting

getPercentInstance

```
public static final NumberFormat getPercentInstance()
```

Returns a percentage format for the current default FORMAT locale.

This is equivalent to calling
`getPercentInstance(Locale.getDefault(Locale.Category.FORMAT))`.

Returns:

the NumberFormat instance for percentage formatting

See Also:

`Locale.getDefault(java.util.Locale.Category)`,
`Locale.Category.FORMAT`

getPercentInstance

```
public static NumberFormat getPercentInstance(Locale inLocale)
```

Returns a percentage format for the specified locale.

Parameters:

inLocale - the desired locale

Returns:

the NumberFormat instance for percentage formatting

getAvailableLocales

```
public static Locale[] getAvailableLocales()
```

Returns an array of all locales for which the `get*Instance` methods of this class can return localized instances. The returned array represents the union of locales supported by the Java runtime and by installed `NumberFormatProvider` implementations. It must contain at least a `Locale` instance equal to `Locale.US`.

Returns:

An array of locales for which localized NumberFormat instances are available.

hashCode

```
public int hashCode()
```

Overrides hashCode.

Overrides:

`hashCode` in class `Object`

Returns:

a hash code value for this object.

See Also:

Object.equals(java.lang.Object),
System.identityHashCode(java.lang.Object)

equals

```
public boolean equals(Object obj)
```

Overrides equals.

Overrides:

equals in class Object

Parameters:

obj - the reference object with which to compare.

Returns:

true if this object is the same as the obj argument; false otherwise.

See Also:

Object.hashCode(), HashMap

clone

```
public Object clone()
```

Overrides Cloneable.

Overrides:

clone in class Format

Returns:

a clone of this instance.

See Also:

Cloneable

isGroupingUsed

```
public boolean isGroupingUsed()
```

Returns true if grouping is used in this format. For example, in the English locale, with grouping on, the number 1234567 might be formatted as "1,234,567". The grouping separator as well as the size of each group is locale dependant and is determined by sub-classes of NumberFormat.

Returns:

true if grouping is used; false otherwise

See Also:

`setGroupingUsed(boolean)`

setGroupingUsed

`public void setGroupingUsed(boolean newValue)`

Set whether or not grouping will be used in this format.

Parameters:

`newValue - true if grouping is used; false otherwise`

See Also:

`isGroupingUsed()`

getMaximumIntegerDigits

`public int getMaximumIntegerDigits()`

Returns the maximum number of digits allowed in the integer portion of a number.

Returns:

`the maximum number of digits`

See Also:

`setMaximumIntegerDigits(int)`

setMaximumIntegerDigits

`public void setMaximumIntegerDigits(int newValue)`

Sets the maximum number of digits allowed in the integer portion of a number. maximumIntegerDigits must be ≥ minimumIntegerDigits. If the new value for maximumIntegerDigits is less than the current value of minimumIntegerDigits, then minimumIntegerDigits will also be set to the new value.

Parameters:

`newValue - the maximum number of integer digits to be shown; if less than zero, then zero is used. The concrete subclass may enforce an upper limit to this value appropriate to the numeric type being formatted.`

See Also:

`getMaximumIntegerDigits()`

getMinimumIntegerDigits

```
public int getMinimumIntegerDigits()
```

Returns the minimum number of digits allowed in the integer portion of a number.

Returns:

the minimum number of digits

See Also:

setMinimumIntegerDigits(int)

setMinimumIntegerDigits

```
public void setMinimumIntegerDigits(int newValue)
```

Sets the minimum number of digits allowed in the integer portion of a number. minimumIntegerDigits must be ≤ maximumIntegerDigits. If the new value for minimumIntegerDigits exceeds the current value of maximumIntegerDigits, then maximumIntegerDigits will also be set to the new value

Parameters:

newValue - the minimum number of integer digits to be shown; if less than zero, then zero is used. The concrete subclass may enforce an upper limit to this value appropriate to the numeric type being formatted.

See Also:

getMinimumIntegerDigits()

getMaximumFractionDigits

```
public int getMaximumFractionDigits()
```

Returns the maximum number of digits allowed in the fraction portion of a number.

Returns:

the maximum number of digits.

See Also:

setMaximumFractionDigits(int)

setMaximumFractionDigits

```
public void setMaximumFractionDigits(int newValue)
```

Sets the maximum number of digits allowed in the fraction portion of a number. maximumFractionDigits must be ≥ minimumFractionDigits. If the new value for maximumFractionDigits is less than the current value of minimumFractionDigits, then minimumFractionDigits will also be set to the new value.

Parameters:

`newValue - the maximum number of fraction digits to be shown; if less than zero, then zero is used. The concrete subclass may enforce an upper limit to this value appropriate to the numeric type being formatted.`

See Also:

`getMaximumFractionDigits()`

getMinimumFractionDigits

```
public int getMinimumFractionDigits()
```

Returns the minimum number of digits allowed in the fraction portion of a number.

Returns:

`the minimum number of digits`

See Also:

`setMinimumFractionDigits(int)`

setMinimumFractionDigits

```
public void setMinimumFractionDigits(int newValue)
```

Sets the minimum number of digits allowed in the fraction portion of a number. minimumFractionDigits must be ≤ maximumFractionDigits. If the new value for minimumFractionDigits exceeds the current value of maximumFractionDigits, then maximumIntegerDigits will also be set to the new value

Parameters:

`newValue - the minimum number of fraction digits to be shown; if less than zero, then zero is used. The concrete subclass may enforce an upper limit to this value appropriate to the numeric type being formatted.`

See Also:

getMinimumFractionDigits()

getCurrency

public Currency getCurrency()

Gets the currency used by this number format when formatting currency values. The initial value is derived in a locale dependent way. The returned value may be null if no valid currency could be determined and no currency has been set using setCurrency.

The default implementation throws UnsupportedOperationException.

Returns:

the currency used by this number format, or null

Throws:

UnsupportedOperationException - if the number format class doesn't implement currency formatting

Since:

1.4

setCurrency

public void setCurrency(Currency currency)

Sets the currency used by this number format when formatting currency values. This does not update the minimum or maximum number of fraction digits used by the number format.

The default implementation throws UnsupportedOperationException.

Parameters:

currency - the new currency to be used by this number format

Throws:

UnsupportedOperationException - if the number format class doesn't implement currency formatting

NullPointerException - if currency is null

Since:

1.4

getRoundingMode

```
public RoundingMode getRoundingMode()
```

Gets the RoundingMode used in this NumberFormat. The default implementation of this method in NumberFormat always throws UnsupportedOperationException. Subclasses which handle different rounding modes should override this method.

Returns:

The RoundingMode used for this NumberFormat.

Throws:

UnsupportedOperationException - The default implementation always throws this exception

Since:

1.6

See Also:

setRoundingMode(RoundingMode)

setRoundingMode

```
public void setRoundingMode(RoundingMode roundingMode)
```

Sets the RoundingMode used in this NumberFormat. The default implementation of this method in NumberFormat always throws UnsupportedOperationException. Subclasses which handle different rounding modes should override this method.

Parameters:

roundingMode - The RoundingMode to be used

Throws:

UnsupportedOperationException - The default implementation always throws this exception

NullPointerException - if roundingMode is null

Since:

1.6

See Also:

C

```
getRoundingMode()
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.